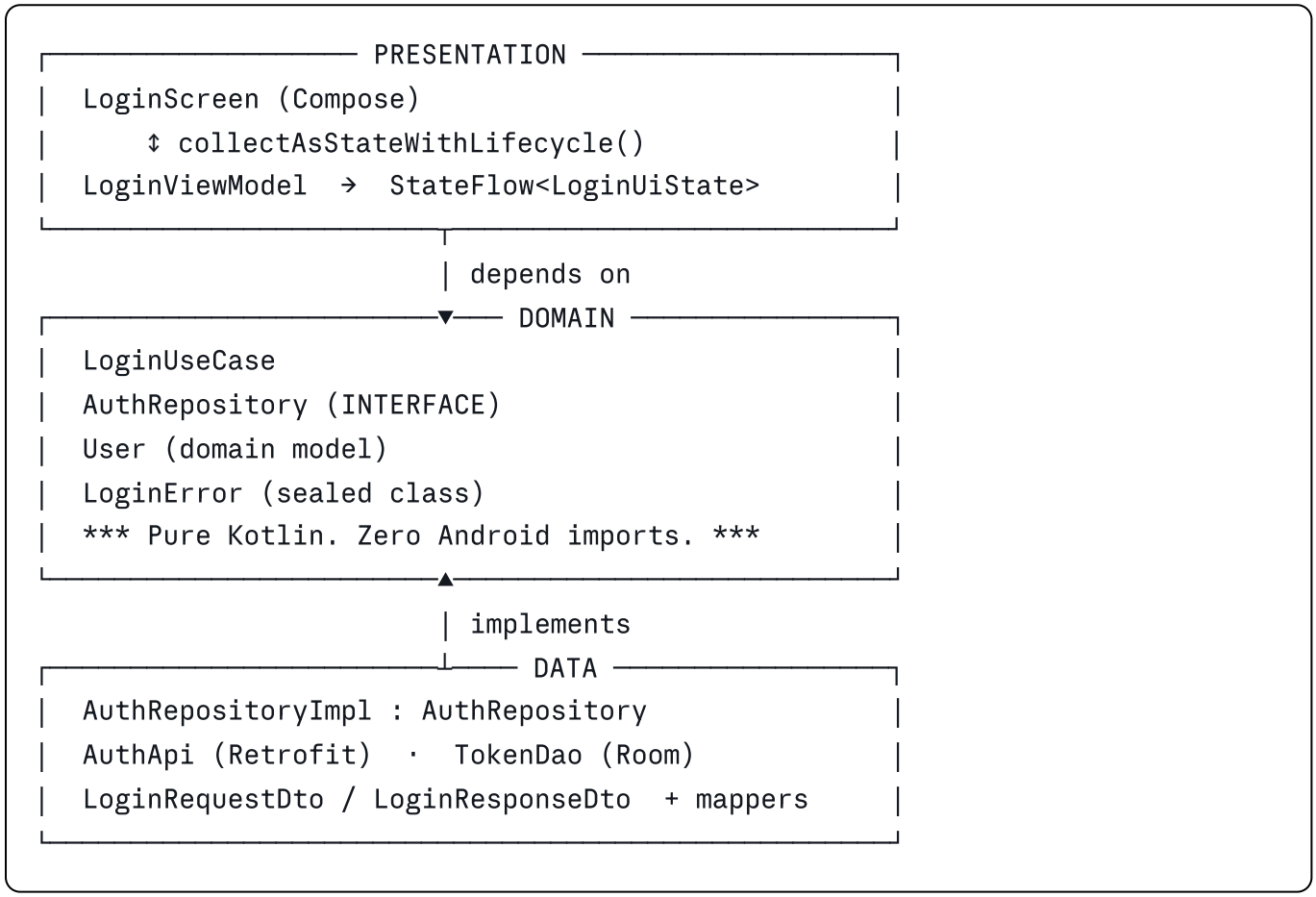
MVVM + Clean Architecture — Login Screen (Interview Notes)

1. The 30-Second Interview Opener

> "I structure apps with MVVM inside Clean Architecture — three layers: Presentation, Domain, and Data. The Domain layer is pure Kotlin with no Android dependencies. It owns the business rules and defines the Repository *interface*. The Data layer implements that interface. The Presentation layer (ViewModel + Compose) depends only on Domain. So dependencies always point inward toward Domain — that's the dependency rule. Let me walk you through it with a login screen."

---

2. Layer Diagram



Dependency rule: Data → Domain ← Presentation. Nothing points *out* of Domain.

---

3. Why Clean Architecture over plain MVVM

Plain MVVM	MVVM + Clean
`View ← ViewModel ← Repository/API`	`View ← ViewModel ← UseCase ← Repository ← DataSource`
ViewModel holds business logic → bloated	Business logic lives in UseCase → thin ViewModel
ViewModel tied to Retrofit/Room types	ViewModel only knows domain models
Hard to unit test (needs Android mocks)	Domain is pure Kotlin → JUnit, no Robolectric
Swapping API for GraphQL touches ViewModel	Only Data layer changes

4. DOMAIN LAYER (write this first in an interview)

4.1 Domain model

kotlin

```kotlin
data class User(
    val id: String,
    val email: String,
    val displayName: String
)
```

4.2 Typed errors (sealed class — this impresses interviewers)

kotlin

```kotlin
sealed class LoginError {
    data object NoInternet : LoginError()
    data object InvalidCredentials : LoginError()
    data object ServerError : LoginError()
    data object Timeout : LoginError()
    data class Validation(val field: Field, val reason: String) : LoginError()
        enum class Field { EMAIL, PASSWORD }
    }
    data object Unknown : LoginError()
}
```

4.3 Result wrapper

kotlin

```kotlin
sealed class Result<out T> {
    data class Success<T>(val data: T) : Result<T>()
    data class Failure(val error: LoginError) : Result<Nothing>()
}
```

4.4 Repository INTERFACE (lives in Domain, not Data)

kotlin

```kotlin
interface AuthRepository {
    suspend fun login(email: String, password: String): Result<User>
    suspend fun saveSession(user: User, token: String)
}
```

> **Key talking point:** The interface lives in Domain because Domain *owns the contract*. Data layer merely satisfies it. This is Dependency Inversion — the "D" in SOLID.

4.5 UseCase — where business rules live

kotlin

```kotlin
class LoginUseCase @Inject constructor(
    private val authRepository: AuthRepository
) {
    suspend operator fun invoke(email: String, password: String): Result<User>

        // Business rule 1: validation happens BEFORE hitting the network
        if (email.isBlank() || !email.contains("@")) {
            return Result.Failure(
                LoginError.Validation(LoginError.Validation.Field.EMAIL, "Enter
            )
        }
        if (password.length < 8) {
            return Result.Failure(
                LoginError.Validation(LoginError.Validation.Field.PASSWORD, "Mi
            )
        }

        // Business rule 2: delegate to repository
        return authRepository.login(email.trim().lowercase(), password)
    }
}
```

Why `operator fun invoke`**?** Lets you call it as `loginUseCase(email, password)` — reads naturally and signals "this class does exactly one thing."

---

5. DATA LAYER

5.1 DTOs (network shapes — never leave the Data layer)

kotlin

```kotlin
data class LoginRequestDto(val email: String, val password: String)

data class LoginResponseDto(
    val userId: String,
    val email: String,
    val name: String,
    val accessToken: String
)
```

5.2 Mapper (DTO → Domain model)

kotlin

```kotlin
fun LoginResponseDto.toDomain() = User(
    id = userId,
    email = email,
    displayName = name
)
```

> **Talking point:** Mappers keep Retrofit/Room annotations out of the Domain. If the API renames `name` to `full_name`, only the DTO and mapper change.

5.3 Retrofit API

kotlin

```kotlin
interface AuthApi {
    @POST("auth/login")
    suspend fun login(@Body request: LoginRequestDto): LoginResponseDto
}
```

5.4 Repository implementation — error translation happens HERE

kotlin

```kotlin
class AuthRepositoryImpl @Inject constructor(
    private val api: AuthApi,
    private val sessionStore: SessionStore
) : AuthRepository {

    override suspend fun login(email: String, password: String): Result<User> =
        withContext(Dispatchers.IO) {
            try {
                val response = api.login(LoginRequestDto(email, password))
                sessionStore.saveToken(response.accessToken)
                Result.Success(response.toDomain())

            } catch (e: UnknownHostException) {
                Result.Failure(LoginError.NoInternet)
            } catch (e: SocketTimeoutException) {
                Result.Failure(LoginError.Timeout)
            } catch (e: HttpException) {
                when (e.code()) {
                    401 -> Result.Failure(LoginError.InvalidCredentials)
                    in 500..599 -> Result.Failure(LoginError.ServerError)
                    else -> Result.Failure(LoginError.Unknown)
                }
            } catch (e: CancellationException) {
                throw e   // NEVER swallow cancellation — rethrow it
            } catch (e: Exception) {
                Result.Failure(LoginError.Unknown)
            }
        }

    override suspend fun saveSession(user: User, token: String) {
        sessionStore.save(user.id, token)
    }
}
```

> **Two things interviewers look for here:**
>
> 1. Raw HTTP codes are translated into *domain* errors — the ViewModel never sees `HttpException`.
> 2. `CancellationException` is rethrown, not swallowed. Swallowing it breaks structured concurrency.

---

6. PRESENTATION LAYER

6.1 UI State — single source of truth

kotlin

```kotlin
data class LoginUiState(
    val email: String = "",
    val password: String = "",
    val isLoading: Boolean = false,
    val emailError: String? = null,
    val passwordError: String? = null,
    val generalError: String? = null,
    val isLoginSuccessful: Boolean = false
) {
    // Derived state — no separate variable needed
    val isLoginEnabled: Boolean
        get() = email.isNotBlank() && password.isNotBlank() && !isLoading
}
```

> **Talking point:** Credentials live in the *ViewModel*, not in Compose `remember`. That's what makes them survive rotation and enables retry without re-typing.

6.2 ViewModel

kotlin

```kotlin
@HiltViewModel
class LoginViewModel @Inject constructor(
    private val loginUseCase: LoginUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow(LoginUiState())
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()

    private var loginJob: Job? = null

    fun onEmailChange(value: String) {
        _uiState.update { it.copy(email = value, emailError = null, generalErro
    }

    fun onPasswordChange(value: String) {
        _uiState.update { it.copy(password = value, passwordError = null, gener
    }

    fun login() {
        // Guard: ignore taps while a request is in flight
        if (_uiState.value.isLoading) return

        // Cancel any previous attempt — latest request wins
        loginJob?.cancel()

        loginJob = viewModelScope.launch {
            _uiState.update {
                it.copy(isLoading = true, emailError = null,
                        passwordError = null, generalError = null)
            }

            when (val result = loginUseCase(_uiState.value.email, _uiState.valu
                is Result.Success -> _uiState.update {
                    it.copy(isLoading = false, isLoginSuccessful = true)
                }
                is Result.Failure -> _uiState.update {
                    it.copy(isLoading = false).applyError(result.error)
                }
            }
        }
    }

    fun retry() = login()   // Credentials still in state — nothing to re-enter

    fun onNavigatedToHome() {
        _uiState.update { it.copy(isLoginSuccessful = false) }  // Consume the
```

```
    }

    // Map domain errors → user-facing strings. Server text is NEVER shown raw.
    private fun LoginUiState.applyError(error: LoginError): LoginUiState = when
        is LoginError.Validation -> when (error.field) {
            LoginError.Validation.Field.EMAIL    -> copy(emailError = error.rea
            LoginError.Validation.Field.PASSWORD -> copy(passwordError = error.
        }
        LoginError.NoInternet         -> copy(generalError = "No internet conne
        LoginError.InvalidCredentials -> copy(generalError = "Incorrect email o
        LoginError.ServerError        -> copy(generalError = "Something went wr
        LoginError.Timeout            -> copy(generalError = "Request timed out
        LoginError.Unknown            -> copy(generalError = "Unexpected error.
    }
}
```

6.3 Compose UI

kotlin

```
@Composable
fun LoginScreen(
    viewModel: LoginViewModel = hiltViewModel(),
    onLoginSuccess: () -> Unit
) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()

    // One-shot navigation event
    LaunchedEffect(state.isLoginSuccessful) {
        if (state.isLoginSuccessful) {
            onLoginSuccess()
            viewModel.onNavigatedToHome()
        }
    }

    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        OutlinedTextField(
            value = state.email,
            onValueChange = viewModel::onEmailChange,
            label = { Text("Email") },
            isError = state.emailError != null,
            supportingText = { state.emailError?.let { Text(it) } },
            singleLine = true,
            enabled = !state.isLoading,
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(Modifier.height(12.dp))

        OutlinedTextField(
            value = state.password,
            onValueChange = viewModel::onPasswordChange,
            label = { Text("Password") },
            visualTransformation = PasswordVisualTransformation(),
            isError = state.passwordError != null,
            supportingText = { state.passwordError?.let { Text(it) } },
            singleLine = true,
            enabled = !state.isLoading,
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(Modifier.height(20.dp))
```

```kotlin
        Button(
            onClick = viewModel::login,
            enabled = state.isLoginEnabled,
            modifier = Modifier.fillMaxWidth()
        ) {
            if (state.isLoading) {
                CircularProgressIndicator(
                    modifier = Modifier.size(20.dp),
                    strokeWidth = 2.dp,
                    color = MaterialTheme.colorScheme.onPrimary
                )
            } else {
                // Button text flips to "Retry" after a failure
                Text(if (state.generalError != null) "Retry" else "Login")
            }
        }

        state.generalError?.let { message ->
            Spacer(Modifier.height(12.dp))
            Text(
                text = message,
                color = MaterialTheme.colorScheme.error,
                style = MaterialTheme.typography.bodyMedium
            )
        }
    }
}
```

---

7. Hilt Wiring

kotlin

```
@Module
@InstallIn(SingletonComponent::class)
abstract class AuthModule {

    @Binds
    @Singleton
    abstract fun bindAuthRepository(impl: AuthRepositoryImpl): AuthRepository
}

@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {

    @Provides
    @Singleton
    fun provideAuthApi(retrofit: Retrofit): AuthApi = retrofit.create(AuthApi::
}
```

> `@Binds` for interface→implementation (cheaper, generates less code). `@Provides` when you must *construct* the object (Retrofit, Room, third-party classes).

---

8. EDGE CASES — the part interviewers actually probe

Edge Case 1 — Network timeout / slow response

- ViewModel stays in `isLoading = true`; Compose shows a spinner **inside** the button.
- Login button disabled via `isLoginEnabled` → user can't spam it.
- Text fields disabled so credentials can't change mid-flight.
- If the user navigates away: Activity destroyed → `viewModelScope` cancelled → the Retrofit call is cancelled by structured concurrency. **No leak, no callback into a dead UI.**

Edge Case 2 — Concurrent / duplicate requests

- First guard: `if (_uiState.value.isLoading) return`.
- Second guard: `loginJob?.cancel()` before launching — **latest request wins.**
- Why it matters: duplicate requests waste server capacity and can produce a race where an older, slower response overwrites a newer one.

Edge Case 3 — Configuration change (rotation)

- ViewModel survives rotation (retained via `ViewModelStore`).
- The coroutine is in `viewModelScope`, not tied to the Activity → keeps running.
- New Composable re-collects the same `StateFlow` → spinner reappears correctly.
- **Critical:** email/password are in `LoginUiState` inside the ViewModel, *not* in `remember { mutableStateOf("") }`. If they were in `remember`, rotation would wipe them.

Edge Case 4 — Error handling and retry

- Errors are translated in **two stages**: Data maps HTTP → `LoginError`; ViewModel maps `LoginError` → user-facing string. The raw server message is never displayed.
- Field-specific errors (`emailError`, `passwordError`) render under the relevant text field.
- Transport errors (`generalError`) render below the button.
- Button label flips to **"Retry"** when `generalError != null`.
- Credentials are still in state → retry is a single tap, no re-typing.
- On retry, `login()` clears all error fields **before** the call, so a stale error never flashes alongside the new spinner.

Edge Case 5 — Success event fired twice

- `isLoginSuccessful` is a state flag, and state *replays* on recomposition/rotation.
- Without consuming it, rotating on the success frame would navigate twice.
- Fix: `LaunchedEffect` navigates, then immediately calls `onNavigatedToHome()` to reset the flag.
- Alternative: a `Channel`/`SharedFlow` for one-shot events instead of a state flag.

---

9. Testability — the payoff

kotlin

```kotlin
class LoginUseCaseTest {

    private val repository = mockk<AuthRepository>()
    private val useCase = LoginUseCase(repository)

    @Test
    fun `rejects short password without calling repository`() = runTest {
        val result = useCase("user@test.com", "123")

        assertTrue(result is Result.Failure)
        coVerify(exactly = 0) { repository.login(any(), any()) }
    }
}
```

No Android runtime. No Robolectric. Plain JUnit. That is the whole point of keeping Domain free of Android imports — say this out loud in the interview.

---

10. Rapid-Fire Answers

Question	Answer
Why does the Repository *interface* live in Domain?	Dependency Inversion. Domain owns the contract; Data satisfies it. Lets you swap REST for GraphQL without touching Domain or Presentation.
Is a UseCase worth it for a one-line pass-through?	Yes — it's the seam where validation, caching policy, and multi-repository orchestration land later. Consistency beats a saved file.
Where does input validation go?	Business validation in the UseCase. Purely cosmetic formatting can stay in the UI.
StateFlow or LiveData?	StateFlow — pure Kotlin, works in Domain, richer operators. Use `collectAsStateWithLifecycle()` to get LiveData's lifecycle safety.
Why not put the API call directly in the ViewModel?	Couples Presentation to Retrofit, makes it untestable without mocking Android, and duplicates logic across screens that share the same call.
How do you handle one-shot events (navigation, toast)?	Either a consumed state flag reset after handling, or a `Channel`/`SharedFlow`. Never a plain state boolean left unreset.
What survives rotation?	ViewModel and everything inside `LoginUiState`. Compose `remember` does **not**.